PIEW ARCHITECTURE SPECIFICATION • RFC-008

# RANCANG BANGUN ARSITEKTUR PIEW AUTONOMOUS SUPERVISOR

Spesifikasi Sistem Rekayasa: Pengawas Mutu Otonom, Protokol Audit Empiris, dan Kemudi Revisi AI-to-AI

| | | | |
|---|---|---|---|
| **Nama Dokumen:** | PIEW-SPEC-SUPERVISOR-V1 | **Klasifikasi:** | Internal Architecture Core |
| **Target Modul:** | packages/prompt-engine | **Status Standar:** | APPROVED FOR ENGINEERING |
| **Protokol:** | MCP / ReAct Empirical Loop | **Tanggal Rilis:** | Oktober 2026 |

## 1. RINGKASAN EKSEKUTIF DAN REPOSISI STRATEGIS

Tujuan fundamental dari arsitektur PIEW bukan untuk bersaing menjadi generator kode generik (seperti Cursor, Windsurf, atau Claude Code), melainkan bertindak sebagai **Insinyur Pengawas Mutu dan Pengarah Otonom (Autonomous Lead Architect & Quality Gatekeeper)**. Dalam ekosistem rekayasa perangkat lunak modern, agen koding bertindak sebagai pekerja pelaksana (*implementor*), sementara PIEW bertindak sebagai mandor pengawas yang memastikan setiap instruksi yang diterjemahkan dan setiap baris kode yang dihasilkan memenuhi standar arsitektur industri, bebas cacat sintaksis, dan aman dari halusinasi fungsional.

> **REPOSISI NILAI UTAMA PIEW**
>
> PIEW tidak mengambil alih penulisan ribuan baris kode aplikasi dari nol. PIEW mengawal seluruh siklus kerja agen koding: mulai dari formalisasi kontrak kebutuhan pengguna, pengawasan langsung di terminal/git/docker, pengujian bukti fisik nyata, hingga memaksakan revisi taktis terarah sampai hasil akhir terbukti 100% akurat.

Dengan memposisikan PIEW sebagai Supervisor Otonom yang memiliki akses alat riil (*terminal, filesystem, git, dan docker*), platform PIEW menyelesaikan persoalan terbesar industri kecerdasan buatan saat ini: *ketidakmampuan pengguna awam maupun pengembang dalam memverifikasi kebenaran hasil kerja AI koding secara objektif*.

## 2. EMPAT PRINSIP DASAR REKAYASA (GOLDEN GOVERNANCE PRINCIPLES)

Arsitektur pengawas otonom PIEW dibangun di atas 4 pilar tata kelola mutlak yang wajib dipatuhi oleh sistem:

| Prinsip Rekayasa | Masalah di Industri Saat Ini | Solusi Mekanis PIEW Supervisor |
|---|---|---|
| **1. Evidence Over Claims (Audit Bukti Nyata)** | Agen koding sering melapor bahwa tugas "sudah selesai 100%", padahal hanya menyertakan komentar `// TODO` atau fungsi semu. | PIEW dilarang mempercayai teks laporan agen. Kelulusan hanya diberikan berdasarkan bukti fisik: git diff, exit code build = 0, dan pengujian unit lulus. |
| **2. Bounded Iteration (Batas Anggaran Putaran)** | Loop perbaikan tanpa batas menyebabkan kebuntuan logika, token exhaustion, dan pembengkakan biaya API secara eksponensial. | Menerapkan *Circuit Breaker* ketat: maksimal 3 kali siklus revisi per fase. Jika batas tercapai, sistem melakukan eskalasi ke operator manusia. |
| **3. Progressive Task Phasing (Dekonstruksi Bertahap)** | Instruksi monolitik besar membuat agen koding kehilangan konteks, menghasilkan kode terpotong, dan gagal mengintegrasikan sistem. | PIEW mendekonstruksi proyek menjadi tahapan linier (Database → API → UI). Agen dilarang menyentuh fase lanjutan sebelum fase aktif lulus uji. |
| **4. Zero Secret Leakage (Keamanan Sandbox)** | Kunci API dan token kredensial rawan bocor ke commit log repositori publik saat agen koding melakukan eksekusi otomatis. | Audit dijalankan pada sandbox lokal berizin terbatas. Kredensial disuntikkan secara bertopeng (masked) melalui variabel lingkungan runtime. |

## 3. ALUR KERJA TERPADU (END-TO-END ORCHESTRATION PIPELINE)

Alur orkestrasi antara Pengguna, PIEW Supervisor, dan Agen Koding Eksternal dijalankan melalui 8 tahapan terstruktur yang bersifat linier dan deterministik:



### Rincian Pelaksanaan Setiap Tahapan:

1. **Tahap 1 — Pre-Flight Ingestion & Kontrak Kerja**: PIEW menerima prompt pengguna, membedah tujuan inti, mengidentifikasi arketipe tugas, menyusun batasan mutlak (*negative constraints*), dan menetapkan kriteria kelulusan formal (*Definition of Done / DoD*).
2. **Tahap 2 — Milestone Phasing**: Tugas besar dipecah menjadi unit-unit logis berurutan. Misalnya pada aplikasi e-commerce: Fase 1 adalah skema database Prisma, Fase 2 adalah endpoint transaksi & pembayaran, Fase 3 adalah antarmuka web.
3. **Tahap 3 — Delegasi Presisi**: PIEW mengirimkan paket instruksi fase aktif ke agen koding (via MCP Server atau prompt terstruktur) lengkap dengan batasan teknologi dan larangan mutlak.
4. **Tahap 4 — Pengerjaan Agen Koding**: Agen pelaksana (seperti Cursor atau Claude Code) memproses instruksi dan melakukan modifikasi berkas di lingkungan lokal.
5. **Tahap 5 — Active Empirical Inspection**: Segera setelah agen koding menyatakan selesai, PIEW Supervisor secara mandiri menjalankan serangkaian alat inspeksi fisik di terminal tanpa meminta konfirmasi dari agen koding.
6. **Tahap 6 — Quality Gate Decision**: Menilai matriks temuan audit ke dalam 3 tingkat keparahan: *Critical* (wajib revisi), *Major* (wajib revisi), atau *Minor* (peringatan).
7. **Tahap 7 — Automated Steering Loop**: Jika ditemukan galat, PIEW merumuskan prompt koreksi taktis yang spesifik menyebutkan nama berkas, nomor baris, pesan galat compiler, dan langkah perbaikan yang ditargetkan.
8. **Tahap 8 — Phase Sign-Off & Delivery**: Fase aktif dinyatakan selesai secara permanen. Progres dikunci di basis data riwayat, dan sistem beralih ke fase berikutnya hingga seluruh proyek tuntas.

## 4. TAKSONOMI ALAT KERJA SUPERVISOR (SUPERVISOR TOOLSET)

Agar mampu melakukan pengawasan secara objektif tanpa halusinasi, runtime PIEW Supervisor dibekali 5 alat kerja kanonis (*Canonical Verification Tools*):

| Nama Tool | Target & Parameter | Metode Kerja Empiris | Pencegahan Kegagalan |
|---|---|---|---|
| `git_inspect_diff` | `path?: string`<br>`targetCommit: string` | Mengeksekusi `git diff` secara terisolasi untuk mendeteksi penambahan, modifikasi, atau penghapusan baris kode. | Mendeteksi bila agen koding menghapus file konfigurasi penting secara tidak sengaja. |
| `terminal_run_verification` | `command: string`<br>`timeoutSec: number` | Menjalankan compiler TypeScript (`tsc --noEmit`) atau test runner (`npm test`) dan menangkap exit code, stdout, serta stderr. | Dilengkapi timeout guard 60 detik untuk mencegah proses terminal membeku selamanya. |
| `docker_health_check` | `containerName: string`<br>`expectedPorts: int[]` | Memeriksa status kontainer Docker lokal (running/restarting/exited), memeriksa pemetaan port, dan membaca 50 baris log terakhir. | Mendeteksi kondisi *crash-loop* pada database atau service latar belakang. |
| `spec_compliance_evaluator` | `taskSpecId: string`<br>`changedFiles: string[]` | Mencocokkan berkas yang diubah terhadap daftar batasan negatif dan persyaratan fungsional dari Task Specification Map. | Menolak hasil jika agen membuat modul yang secara eksplisit dilarang oleh pengguna. |
| `generate_steering_prompt` | `auditFailures: Defect[]`<br>`phaseId: string` | Mengonversi seluruh log kegagalan teknis menjadi naskah instruksi perbaikan yang terstruktur dan mudah dieksekusi agen koding. | Menghilangkan nada emosional/berbelit-belit; fokus pada perbaikan kode deterministik. |

## 5. KONTRAK DATA DAN SKEMA KEPUTUSAN MUTU (DATA SCHEMAS)

Seluruh komunikasi pengawasan diatur menggunakan skema data validasi Zod ketat yang bertempat di `packages/prompt-contracts/src/supervision.ts`:

### A. Skema Kontrak Pengawasan Fase Proyek (PhasedProjectSessionSchema)

```
export const PhasedProjectSessionSchema = z.object({
  sessionId: z.string().uuid(),
  projectName: z.string().min(1),
  userOriginalPrompt: z.string(),
  totalPhases: z.number().int().min(1),
  currentPhaseIndex: z.number().int().nonnegative(),
  phases: z.array(z.object({
    phaseId: z.string(),
    name: z.string(),
    targetArchetype: z.string(),
    definitionOfDone: z.array(z.string()),
    negativeConstraints: z.array(z.string()),
    retryCount: z.number().int().default(0),
    maxRetries: z.number().int().default(3),
    status: z.enum(["PENDING", "IN_PROGRESS", "AUDITING", "COMPLETED", "HALTED"]),
    auditTrail: z.array(z.string()).default([]),
  })),
  overallStatus: z.enum(["INITIALIZING", "RUNNING", "COMPLETED", "HALTED_MAX_RETRIES"]),
});
```

### B. Skema Hasil Audit Empiris (EmpiricalAuditResultSchema)

```
export const EmpiricalAuditResultSchema = z.object({
  auditId: z.string().uuid(),
  phaseId: z.string(),
  timestamp: z.string().datetime(),
  verificationChecks: z.object({
    typecheckPassed: z.boolean(),
    testSuitePassed: z.boolean(),
    negativeConstraintsRespected: z.boolean(),
    runtimeHealthPassed: z.boolean(),
  }),
  defectsDetected: z.array(z.object({
    severity: z.enum(["CRITICAL", "MAJOR", "MINOR"]),
    fileLocation: z.string().optional(),
    lineRange: z.string().optional(),
    failureMessage: z.string(),
    rawStdoutStderr: z.string().optional(),
  })),
  verdict: z.enum(["APPROVED_PASS", "REJECT_REVISE", "ESCALATE_BLOCKED"]),
  steeringDirective: z.string().optional(),
});
```

## 6. FORMAT RESMI INSTRUKSI KEMUDI REVISI (STEERING PROMPT FORMAT)

Ketika PIEW Supervisor mendeteksi kegagalan audit pada Tahap 5, sistem secara otomatis merumuskan *Steering Prompt* dengan struktur baku 4-komponen yang telah terbukti memiliki tingkat keberhasilan perbaikan tertinggi saat diterima oleh Cursor atau Claude Code:

```
[PIEW SUPERVISOR REJECTION DIRECTIVE — PHASE {PHASE_NUMBER}: {PHASE_NAME}]
Pekerjaan Anda pada fase ini DITOLAK oleh PIEW Quality Gatekeeper karena tidak memenuhi standar verifikas

1. RINGKASAN DEFECT KRITIS:
   - File: {PATH_TO_FILE} (Baris: {LINE_NUMBERS})
   - Kategori Kegagalan: {DEFECT_CATEGORY} (Misal: TypeScript Type Mismatch / Unhandled Promise Rejectior
   - Bukti Output Terminal:
     >>> {RAW_COMPILER_OR_TEST_STDERR}

2. PELANGGARAN BATASAN MUTLAK (JIKA ADA):
   - Anda melanggar batasan pengguna: "{VIOLATED_NEGATIVE_CONSTRAINT}".

3. TINDAKAN PERBAIKAN WAJIB (ACTION REQUIRED):
   - Perbaiki berkas {PATH_TO_FILE} agar tipe data kembali sesuai kontrak.
   - Tambahkan blok penanganan error pada fungsi {FUNCTION_NAME}.
   - Pastikan perintah `npm run build` dan `npm test` menghasilkan Exit Code 0.

4. CATATAN BATAS ANGGARAN:
   - Percobaan ke-{RETRY_CURRENT} dari maksimum {MAX_RETRIES} toleransi perbaikan.
   - Perbaiki berkas tersebut sekarang dan laporkan kembali setelah tuntas.
```

## 7. INTEGRASI MODEL CONTEXT PROTOCOL (MCP SERVER DI IDE)

Pengguna yang menggunakan IDE AI modern (Cursor, Windsurf, atau Claude Code) dapat menghubungkan PIEW Supervisor secara native melalui protokol MCP:

> **SEAMLESS IDE MCP INTEGRATION**
>
> Pengembang cukup menambahkan PIEW ke berkas konfigurasi MCP mereka (`cursor.json` atau `claude_desktop_config.json`). Agen koding di dalam IDE akan melihat PIEW sebagai alat verifikasi otoritatif yang dapat dipanggil sebelum menandai pekerjaan selesai.

Contoh konfigurasi integrasi pada IDE pengembang:

```
{
  "mcpServers": {
    "piew-supervisor": {
      "command": "piew",
      "args": ["mcp"],
      "env": {
        "PIEW_AUTONOMOUS_SUPERVISOR": "true",
        "PIEW_MAX_PHASE_RETRIES": "3"
      }
    }
  }
}
```

## 8. ROADMAP IMPLEMENTASI TEKNIS PADA MONOREPO PIEW

Penerapan rancang bangun ini tidak memerlukan perubahan merusak pada monorepo dan dapat diintegrasikan secara 100% aditif:

- **Fase A (Penyempurnaan Kontrak)**: Memperkaya `packages/prompt-contracts/src/supervision.ts` dengan skema `EmpiricalAuditResultSchema` dan tipe defek teknis.
- **Fase B (Aktivasi Runtime Empiris)**: Menghubungkan fungsi terminal runner lokal dan git parser ke dalam `packages/prompt-engine/src/supervisor.ts`.
- **Fase C (Eksposur Subperintah CLI)**: Menambahkan subperintah `npm run piew -- supervise <prompt>` pada `scripts/piew-cli.ts` untuk pengoperasian langsung dari terminal pengguna.
- **Fase D (Tampilan Status Dashboard)**: Menghubungkan log audit empiris ke panel pengawasan antarmuka web masa depan.